

Overview

The manifest file format is designed to convey the unique information about a group of secure sub-systems, including unique ID (e.g., serial number), public keys and certificates. This was primarily developed for CryptoAuthentication™ (currently ATECC608, ECC204 and ECC206) and CryptoAutomotive™ (currently TA010, TA100 and TA101) secure sub-system devices. However, it is structured to work for other secure sub-systems as well.

Manifest files provide a way to link an actual Microchip Trust secure sub-system device to the infrastructure environment that it needs to connect to. They are an essential component of the Microchip Trust&GO, TrustMANAGER, TrustFLEX and, optionally, TrustCUSTOM development environments. Whether the device connects to an IoT cloud, a Wireless Power Consortium (WPC) Qi® charger, a LoRaWAN® network or another type of infrastructure, the manifest file uniquely associates each secure sub-system device with its designated environment.

When using Microchip Trust&GO, TrustMANAGER, TrustFLEX or TrustCUSTOM products, a manifest file is generated for a group of secure sub-system devices that are provisioned through the Microchip Just-In-Time provisioning services. Each object entry in the manifest file is known as a signed secure sub-system device, authenticated using a Microchip Elliptic Curve Cryptography (ECC) private key. The manifest file comprises multiple signed secure sub-system device entries, each containing manufacturer details, product information and device-specific data.

The manifest file is securely provided only to the customer who orders the group of secure sub-system devices. Access to these files is integrated into the development and provisioning workflow offered by Microchip. Once provisioning is complete, the manifest file is made available for download.

Note: The manifest file may contain information about more secure sub-system devices than those delivered in the customer's order. It may also contain information about secure sub-system devices that were rejected during the Microchip Just-In-Time provisioning process. All rejected devices are securely disposed of during this process. Please contact Microchip sales assistance or product support for further information.

Table of Contents

Overview.....	1
1. Manifest Generation.....	3
1.1. Microchip vs. Self-Generated Files.....	3
1.2. CryptoAuthentication™ Trust&GO vs. TrustMANAGER vs. TrustFLEX vs. TrustCUSTOM Files.....	4
1.3. Prototype vs. Production Device Files.....	4
2. Structure and Format of a Manifest File.....	6
2.1. Introduction.....	6
2.2. Binary Encoding.....	6
2.3. SecureElementManifest Object.....	6
2.4. SignedSecureElement Object.....	6
2.5. SecureElement Object.....	7
2.6. EntityName Object.....	8
2.7. PublicJWK Object.....	8
2.8. ModelInfo Object.....	11
3. Manifest File Example and Decoding.....	13
3.1. Manifest Example.....	13
3.2. Decode Python Example.....	15
4. Revision History.....	19
Microchip Information.....	20
Trademarks.....	20
Legal Notice.....	20
Microchip Devices Code Protection Feature.....	20

1. Manifest Generation

The manifest file for the Trust&GO, TrustMANAGER, TrustFLEX and TrustCUSTOM secure sub-system devices can be generated in two ways:

1. Through Microchip Just-In-Time provisioning services (Microchip-generated), or
2. Via custom generation using the provided scripts (self-generated).

In both cases, the Trust&GO, TrustMANAGER, TrustFLEX and TrustCUSTOM devices will have different information due to differences in their configuration. The following sections outline the key differences between Microchip-generated and self-generated manifest files:

1. Manifest signature
2. CryptoAuthentication™ Trust&GO, TrustMANAGER, TrustFLEX and TrustCUSTOM files
3. Prototype and production device files

1.1. Microchip vs. Self-Generated Files

The manifest file format and generation procedures are publicly available, allowing users to generate manifest files independently. However, even when following the documented procedures, there may be minor differences between Microchip-generated and self-generated files.

Manifest Signature

In the manifest file, each secure sub-system device is signed to ensure content integrity. For a Microchip-generated manifest file, the signing operation is performed by Microchip using its Certificate Authority (CA). The corresponding CA certificate can be downloaded from the Microchip website and can be used to validate the authenticity of the Microchip-generated files.



Tip:

- [MCHP Manifest Signer Certificates](#) are available in the **Documentation** section.

For a self-generated manifest file, it is not possible to have each secure sub-system device signed by the Microchip CA, as users do not have access to a CA private key. Instead, users must generate or use a local CA to perform the signature operations. In this case, users are required to share the validation certificate along with the manifest file to enable others to verify the content before further use.

Other differences include:

1. Trust&GO – Content remains the same, as the secure sub-system device data are immutable, but signature and verification certificates are different, as self-generated scripts use their own CA.
2. TrustFLEX
 - a. Device and signer certificates may differ if custom PKI is selected during resource generation.
 - b. For CryptoAuthentication™ secure sub-system devices, slots 1-4 and 13-15 may vary based on additional key generation performed during resource generation at the user's location.
 - c. Signature and verification certificates will differ, as self-generated scripts use their own CA.



Tip:

- [Trust&GO manifest generation scripts](#)
- [TrustFLEX manifest generation scripts \(with dev key generation\)](#)

1.2. CryptoAuthentication™ Trust&GO vs. TrustMANAGER vs. TrustFLEX vs. TrustCUSTOM Files

The manifest files contain only public information related to the secure sub-system device, such as the serial number, certificates and public information for each slot. Depending on the configuration, the information in Trust&GO, TrustMANAGER, TrustFLEX and TrustCUSTOM files varies as follows:

Trust&GO	TrustMANAGER	TrustFLEX	TrustCUSTOM
- Slot 0 public key information (immutable) - Device and signer certificates signed by Microchip CA (immutable)	Slot 0 public key information (immutable)	- Slot 0 public key information (immutable) - Device and signer certificates signed by Microchip or customer CA based on custom PKI selection - Slot 1-4 public key information - Slot 13-15 public key information	Custom information due to unique configuration

Certificate Slots in TrustFLEX Devices

When the user opts to create a custom certificate chain on a TrustFLEX secure sub-system device, the factory-provisioned certificates are overwritten. The Trust Platform Design Suite scripts/notebook provide the option to back up default certificates to a local folder before overwriting custom certificates on the device. However, if the board changes hands after provisioning, the new user will not have access to the backup certificates and will be unable to restore the factory defaults.

1.3. Prototype vs. Production Device Files

Prototype devices are intended for in-house research and development and, as such, do not include a factory-generated manifest file. During factory provisioning of CryptoAuthentication™ and CryptoAutomotive™ secure sub-system devices, either a Slot 0 key or a device handle and key is generated, along with device and signer certificates. For prototype Trust&GO and TrustFLEX devices, it is necessary to self-generate the manifest files.



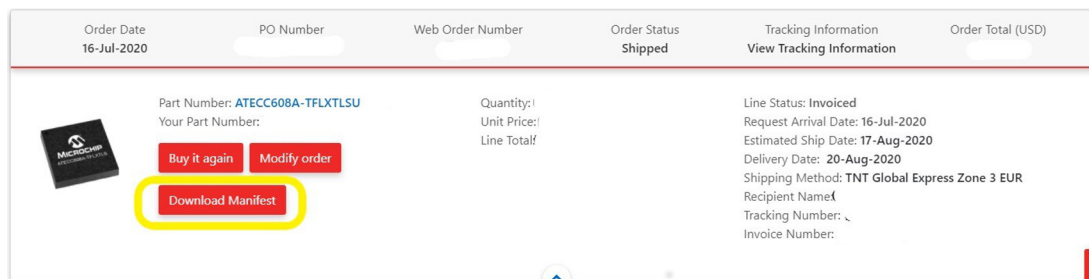
Tip:

- [Trust&GO manifest generation scripts](#)
- [TrustFLEX manifest generation scripts \(with dev key generation\)](#)

For production secure sub-system devices, users can download the manifest file from Microchip using one of the following methods:

microchipDIRECT Manifest Portal

The manifest file can be downloaded from the microchipDIRECT portal under the user's personal login. The manifest file is available for download only after the secure sub-system devices have been provisioned and shipped to the customer.



Microchip SFTP Server Manifest Portal

The manifest file can be downloaded from the Microchip secure file transfer protocol (SFTP) server portal under the user's personal login. The manifest file is available for download only after the secure sub-system devices are provisioned and shipped to the customer. Please contact Microchip sales assistance or product support for more information.

**Tip:**

- [Microchip SFTP server portal](#) (web portal)

Microchip SFTP server connection information

- **Host:** sftp.microchip.com
 - **Port:** 22
-

2. Structure and Format of a Manifest File

2.1. Introduction

The base format is an array of JavaScript Object Notation (JSON) objects. Each object represents a single secure sub-system device and is signed to allow cryptographic verification of its origin. The format is intentionally "flattened," meaning that common information is repeated for each secure sub-system device. This approach facilitates parallel processing of manifest files and allows for the division of entries into smaller manifests when necessary.

This format leverages the JavaScript Object Signing and Encryption (JOSE) standards to represent keys (using JSON Web Key – JWK), certificates (using the x5c member in a JWK) and to provide signing capabilities (using JSON Web Signature – JWS).

Within the object definitions, member values may reference the name of another JSON object or may simply be example values.

2.2. Binary Encoding

JSON does not natively support binary data, so a number of string encodings are used to represent binary data, depending on the context.

BASE64URL	A base64 encoding that uses a URL-safe alphabet, as described in RFC 4648 section 5 , with trailing padding characters ("=") removed. This encoding is used by the JOSE standards and appears in the JWS, JWK and JWE objects, as documented in RFC 7515 section 2 . This encoding is also used in a few other non-JOSE members to maintain consistency.
BASE64URLUINT	A base64url encoding of the value's unsigned big-endian representation as an octet sequence, as described in RFC 4648 section 5 . This encoding is used by the JOSE standards and can be found in the JWK objects used, as documented in RFC 7518 section 2 .
BASE64	The standard base64 encoding, as described in RFC 4648 section 4 , and includes the trailing padding characters ("="). This is used for encoding certificates (JOSE x5c members), to more closely align with the common PEM encoding format used for certificates.
HEX	In some cases, short binary values are expressed as lowercase hex strings. This is to match convention with how these values are typically seen and worked with.

2.3. SecureElementManifest Object

At the top level, the secure sub-system device manifest format is a JSON array of [SignedSecureElement](#) objects where each element represents a single secure sub-system device.

```
[
  SignedSecureElement,
  SignedSecureElement,
  ...
]
```

2.4. SignedSecureElement Object

The SignedSecureElement object is a JWS ([RFC 7515](#)) object using the Flattened JSON Serialization Syntax ([section 7.2.2](#)).

```
{
  "payload": BASE64URL(UTF8(SecureElement)),
  "protected": BASE64URL(UTF8(SignedSecureElementProtectedHeader)),
  "header": {
    "uniqueId": "0123f1822c38dd7a01"
  },
}
```

```

    "signature": BASE64URL(JWS Signature)
  }

```

RFC 7515 section 7.2.1 outlines the encoding and contents of the JWS members used in this object. Below are brief summaries and additional details about these members and the specific features being used.

payload	An encoded SecureElement object, which is the primary content being signed. All information about the secure sub-system device is contained here.
protected	An encoded SignedSecureElementProtectedHeader object, which describes how to verify the signature.
header	The JWS unprotected header. This object contains the unique ID member repeated from the SecureElement object in the payload. The unprotected header is not part of the signed data in the JWS; therefore, it does not need to be encoded and is included to facilitate plain-text searches of the manifest without needing to decode the payload.
signature	The encoded JWS signature of the payload and protected members.

2.4.1. SignedSecureElementProtectedHeader Object

The SignedSecureElementProtectedHeader object is a JWS protected header that describes how to verify the signature. While RFC 7515 section 4.1 outlines the available header members for a JWS, only the members listed below will be used.

```

{
  "alg": "ES256",
  "kid": BASE64URL(Subject Key Identifier),
  "x5t#S256": BASE64URL(SHA-256 Certificate Thumbprint)
}

```

alg	Describes the key type used to sign the payload (see RFC 7518 section 3.1 for additional details). Only public key algorithms will be used.
kid	Encoded Subject Key Identifier (RFC 5280 section 4.2.1.2) of the key used to sign the payload. This is the BASE64URL encoding of the subject key identifier value, not the full extension. This value helps identify the key for verification purposes. Note that while kid is a free-form field in the JWS standard (see RFC 7515 section 4.1.4), this definition applies only to the SignedSecureElement object.
x5t#S256	SHA-256 thumbprint (also known as a fingerprint) of the certificate for the public key required to validate the signature. Like kid, this value can be used to help identify the key for verification (see RFC 7515 section 4.1.8 for further information).

2.5. SecureElement Object

The SecureElement object contains relevant information pertaining to the secure sub-system.

```

{
  "version": 3,
  "model": "ATECC608C",
  "partNumber": "ATECC608C-MAH22",
  "manufacturer": EntityName,
  "provisioner": EntityName,
  "distributor": EntityName,
  "groupId": "359SCE55NV38H3CB",
  "provisioningTimestamp": "2024-01-15T17:22:45.000Z",
  "uniqueId": "0123f1822c38dd7a01",
  "publicKeySet": {
    "keys": [ PublicKeyJWK, ... ]
  },
  "modelInfo": ModelInfo
}

```

version	Integer indicating the version of the SecureElement object. The current version is 3. Subsequent versions will strive to maintain backward compatibility with previous versions, where possible.
----------------	--

model	Name of the base secure sub-system model. The current options for the CryptoAuthentication™ family are ATECC608A, ATECC608B, ATECC608C, ECC204 and ECC206. The current options for the CryptoAutomotive™ family are TA010, TA100 and TA101.
partNumber	Complete part number of the provisioned secure sub-system device.
manufacturer	An EntityName object that identifies the manufacturer of the secure sub-system device.
provisioner	An EntityName object that identifies who performed the provisioning/programming of the secure sub-system device.
distributor	An EntityName object that identifies who distributed the provisioned secure sub-system devices. In many cases, this will be the same entity that generates the manifest data being described here.
groupid	Secure sub-system devices may be organized into groups identified by a single ID. If the secure sub-system device is part of a group, this is the unique ID of that set. Group IDs should be globally unique.
provisioningTimestamp	Date and time the secure sub-system device was provisioned in UTC. Formatting is per RFC 3339 .
uniqueId	Unique identifier for the secure sub-system device. For CryptoAuthentication™ and CryptoAutomotive™ secure sub-system devices, this is the device serial number represented as a lowercase hex string.
publicKeySet	An object representing the public keys (and certificate chains, if available) corresponding to private keys held by the secure sub-system device. This object is a JSON Web Key Set (JWK Set) per RFC 7517 section 5 , where keys are an array of PublicJWK objects.
modelInfo	If additional non-cryptographic information about the secure sub-system device needs to be conveyed, this ModelInfo object may be present with model-specific information.

2.6. EntityName Object

The EntityName object is used to identify an entity responsible for a specific aspect of the secure sub-system device. The members in this object are variable and must be the same as the attributes defined in section 6 of [ITU-T X.520 \(ISO/IEC 9594-6\)](#). While none of the members are required, at least one must be present. The example below shows typical use with Organization Name (6.4.1) and Organizational Unit Name (6.4.2).

```
{
  "organizationName": "Microchip Technology Inc",
  "organizationalUnitName": "Secure Computing Group",
}
```

organizationName	Name of the entity organization (e.g., company name).
organizationalUnitName	Optional name of a unit within the organization to which the entity pertains.

2.7. PublicJWK Object

The PublicJWK object represents an asymmetric public key and any associated certificates. This is a JWK object, as defined in [RFC 7517](#). For convenience, key JWK member specifications are summarized below, along with expectations for specific secure sub-system device models.

The following definitions apply to elliptic curve public keys supported by the CryptoAuthentication™ and CryptoAutomotive™ secure sub-systems. However, the definitions are structured to work for other secure sub-systems as well.

NIST P-224

```
{
  "kid": "0",
  "kty": "EC",
  "crv": "P-224",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
}
```



```

    "x5c": [ BASE64(cert), ... ]
  }

```

NIST P-256

```

{
  "kid": "0",
  "kty": "EC",
  "crv": "P-256",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
  "x5c": [ BASE64(cert), ... ]
}

```

NIST P-384

```

{
  "kid": "0",
  "kty": "EC",
  "crv": "P-384",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
  "x5c": [ BASE64(cert), ... ]
}

```

NIST P-521

```

{
  "kid": "0",
  "kty": "EC",
  "crv": "P-521",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
  "x5c": [ BASE64(cert), ... ]
}

```

Secp256k1 (Bitcoin/Blockchain)

```

{
  "kid": "0",
  "kty": "EC",
  "crv": "secp256k1",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
  "x5c": [ BASE64(cert), ... ]
}

```

Brainpool 256-bit

```

{
  "kid": "0",
  "kty": "EC",
  "crv": "brainpoolP256r1",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
  "x5c": [ BASE64(cert), ... ]
}

```

Brainpool 384-bit

```

{
  "kid": "0",
  "kty": "EC",
  "crv": "brainpoolP384r1",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
  "x5c": [ BASE64(cert), ... ]
}

```

Brainpool 512-bit

```
{
  "kid": "0",
  "kty": "EC",
  "crv": "brainpoolP512r1",
  "x": BASE64URL(x),
  "y": BASE64URL(y),
  "x5c": [ BASE64(cert), ... ]
}
```

The following JWK fields, required for elliptic curve public keys, are defined in [RFC 7518 section 6.2.1](#):

kid	Key ID string that uniquely identifies this key on the secure sub-system device. For CryptoAuthentication™ secure sub-system devices (currently ATECC608, ECC204 and ECC206) and CryptoAutomotive™ secure sub-system devices (currently TA010), this will be the slot number of the corresponding private key. For CryptoAutomotive™ secure sub-system devices (currently TA100 and TA101), this will be the data source name of the corresponding data source defined in the secure sub-system device's secure key provisioning service configuration file.
kty	The EC public key type.
crv	For elliptic curve keys, this is the curve name as defined in RFC 7518 section 6.2.1.1 .
x	For elliptic curve keys, this is the encoded public key X integer, as defined in RFC 7518 section 6.2.1.2 .
y	For elliptic curve keys, this is the encoded public key Y integer, as defined in RFC 7518 section 6.2.1.3 .
x5c	If the public key has an associated certificate, it will appear as the first element in this array. Subsequent certificates in the array will be the CA certificates used to validate the previous one. Certificates will be BASE64 encoded (not BASE64URL) strings of the DER certificate. This is defined in RFC 7517 section 4.7 .

The following definitions apply to octet key pairs supported by the CryptoAutomotive™ secure sub-system devices. However, the definitions are structured to work for other secure sub-systems as well. The JWT octet key pair object is defined in [RFC 8037 section 3.1](#).

Ed25519

```
{
  "kid": "0",
  "kty": "OKP",
  "crv": "Ed25519",
  "x": BASE64URL(x),
  "x5c": [ BASE64(cert), ... ]
}
```

Ed448

```
{
  "kid": "0",
  "kty": "OKP",
  "crv": "Ed448",
  "x": BASE64URL(x),
  "x5c": [ BASE64(cert), ... ]
}
```

X25519

```
{
  "kid": "0",
  "kty": "OKP",
  "crv": "X25519",
  "x": BASE64URL(x),
  "x5c": [ BASE64(cert), ... ]
}
```

X448

```
{
  "kid": "0",

```

```

"ktv": "OKP",
"crv": "X448",
"x": BASE64URL(x),
"x5c": [ BASE64(cert), ... ]
}

```

The following JWK fields, required for octet key pair public keys, are defined in [RFC 8037 section 2](#):

kid	Key ID string that uniquely identifies this key on the secure sub-system device. For CryptoAuthentication™ secure sub-system devices, this will be the slot number of the corresponding private key. For CryptoAutomotive™ secure sub-system devices, this will be the data source name of the corresponding data source defined in the secure sub-system device's secure key provisioning service configuration file.
ktv	The public key type.
crv	The key subtype.
x	The encoded complete public key (curve point (x,y)), as defined in RFC 8032 section 5.1.2 .
x5c	If the public key has an associated certificate, it will appear as the first element in this array. Subsequent certificates in the array will be the CA certificates used to validate the previous one. Certificates will be BASE64 encoded (not BASE64URL) strings of the DER certificate. This is defined in RFC 7517 section 4.7 .

The following definitions apply to RSA public keys supported by the CryptoAutomotive™ secure sub-system devices. However, the definitions are structured to work for other secure sub-systems as well.

RSA

```

{
  "kid": "0",
  "ktv": "RSA",
  "n": BASE64URLUINT(n)
  "e": BASE64URLUINT(e)
  "x5c": [ BASE64(cert), ... ]
}

```

The following JWK fields, required for RSA public keys, are defined in [RFC 7518 section 6.3](#).

kid	Key ID string that uniquely identifies this key on the secure sub-system device. For CryptoAutomotive™ secure sub-system devices, this will be the data source name of the corresponding data source defined in the secure sub-system device's secure key provisioning service configuration file.
ktv	The public key type.
n	The encoded modulus value for the RSA public key, as defined in RFC 7518 section 6.3.1.1 .
e	The encoded exponent value for the RSA public key, as defined in RFC 7518 section 6.3.1.2 .
x5c	If the public key has an associated certificate, it will appear as the first element in this array. Subsequent certificates in the array will be the CA certificates used to validate the previous one. Certificates will be BASE64 encoded (not BASE64URL) strings of the DER certificate. This is defined in RFC 7517 section 4.7 .

2.8. ModelInfo Object

The ModelInfo object holds additional model-specific information about a secure sub-system device that is not captured by the other cryptographic members. It has no specific members, but depends on the model of the secure sub-system device.

At this time, the ModelInfo object is defined only for the CryptoAuthentication™ ATECC608 model.

2.8.1. CryptoAuthentication™ ModelInfo Object

ModelInfo members defined for the CryptoAuthentication™ ATECC608 model:

```

{
  "deviceRevision": "00006002",

```

```
"publicData": [ CryptoAuthPublicDataElement, ... ]
}
```

deviceRevision	The 4-byte device revision number, as returned by the <code>Info</code> (Mode = 0x00) command. This value is encoded as a lowercase hex string.
publicData	An array of CryptoAuthPublicDataElement objects that defines a location and the public data at that location.

2.8.1.1. CryptoAuthPublicDataElement Object

This object defines the location and contents of a public data element in the CryptoAuthentication™ secure sub-systems.

```
{
  "zone": "data",
  "slot": 14,
  "offset": 0,
  "data": BASE64URL(data)
}
```

zone	CryptoAuthentication™ zone where the data are found. The options are “data” for one of the slots, “otp” for the OTP zone or “config” for the configuration zone.
slot	If the zone is “data”, this is the slot index (0-15) where the data can be found.
offset	Byte offset into the zone/slot where the data can be found.
data	Actual data at the location specified by the other members. This data will be BASE64URL encoded, and any padding characters (“=”) will be removed.

Family Reference Manual
© 2022-2025 Microchip Technology Inc. and its subsidiaries

```
{
  "typ": "JWT",
  "alg": "ES256",
  "kid": "TCnnpnzYB8M3C0776ByPJ2yX3KY0",
  "x5t#s256": "Vx2ZhBEX46-zYtiIYEWQP6eFzCg90ugGjh5epGBM4OM"
}
```

```

{
  "version": 3,
  "model": "ATECC608C",
  "partNumber": "ATECC608C-MAH22",
  "manufacturer": {
    "organizationName": "Microchip Technology Inc",
    "organizationalUnitName": "Secure Computing Group"
  },
  "provisioner": {
    "organizationName": "Microchip Technology Inc",
    "organizationalUnitName": "Secure Computing Group"
  },
  "distributor": {
    "organizationName": "Microchip Technology Inc",
    "organizationalUnitName": "Microchip Direct"
  },
  "groupId": "359SCE55NV38H3CB",
  "provisioningTimestamp": "2024-01-15T17:22:45.000Z",
  "uniqueId": "0123f1822c38dd7a01",
  "publicKeySet": {
    "keys": [
      {
        "kid": "0",
        "kty": "EC",
        "crv": "P-256",
        "x": "x8TPQk7h5Ow-cb15p-TE6IRqHQSEtpRNNbu7n10FowM",
        "y": "ux3uP8AloUm8QnNnyFL6R0KKZYxFCItU_QLgguhXoos",
        "x5c": [

```

```

    "kid": "3",
    "kty": "EC",
    "crv": "P-256",
    "x": "jKB8Dkci5ExRzg0qtDdAjpRIHShxYON82YZ2-hajenY",
    "y": "NYMJ9DtbCt6Opnj2g43Ahk2pxQu9KRdMy3m0f-J_rRE"
  },
  {
    "kid": "4",
    "kty": "EC",
    "crv": "P-256",
    "x": "LTU0IGh3ymAzWlWmZ84fhX7Ykb4ZCmmlV-YONdtDaDU",
    "y": "cvNr2TJDWXf4XO6PzybRWoEcQLT4F3NVP8Yj2-X8qbW"
  }
]
}
}
]

```

The SignedSecureElement example above can be verified with the following certificate:

```

-----BEGIN CERTIFICATE-----
MIIB0jCCAXigAwIBAgIQcPl4hLlwUpir3i8tdzrmhTAKBggqhkJOPQQDAjBBMSEw
HwYDVQQKBhNaWNYb2NoaXAgaGVGvjaG5vbG9neSBjbmMxHDAaBgNVBAMME0lhbmlm
ZXN0IFNpZ251ciAwMDUwIBoNMjQwMzA3MDEzMjI4WhgPOTk5OTEyMzEyMzU5NTla
MEEeITAFBgNVBAoMGElpY3JvY2hpcCBUZWNoZm9sb2d5IEluYzEcMBoGA1UEAwWT
TWFuawZlc3QGU2lnbmVvIDAwNTBZMBMGByqGSM49AgEGCCqGSM49AwEHA0IABEYv
Hdl9eahGVVB6qBk10hJJo0LQYVbp+WiF7bINiHwYjgaDbaxicTYUmhxegfq16nMM
3Wb8C5jfszBG3JIj5JiJUDBOMB0GA1UdDgQWBBRMKemfNgHwzcI7vvoHI8nbJfcp
ijAfBgNVHSMEGDAwGBRMKemfNgHwzcI7vvoHI8nbJfcpijAMBQNVHRMBAf8EAjAA
MAoGCCqGSM49BAMCA0gAMEUCIEqosgzOcf4YutuSZbw4d3CunMmME4pYUIMhQCKG
9aHnAiEAzjNY6YJxtgEwzzgBumXPlA5IS7EnUQCbcLkGUDh3uMM=
-----END CERTIFICATE-----

```

3.2. Decode Python Example

The following Python script demonstrates how to verify the signed entries and decode their contents. The script has been tested on Python v3.9.10. Required packages can be installed using the Python package manager, pip:

```
pip install python-jose[cryptography]
```

```

# (c) 2024 Microchip Technology Inc. and its subsidiaries.
#
# Subject to your compliance with these terms, you may use this Microchip software
# and any derivatives of this software. You must retain the above copyright notice
# with any redistribution of this software and the following disclaimers. It is
# your responsibility to comply with third party license terms applicable to your
# use of third party software (including open source software) that may accompany
# this Microchip software.
#
# THIS SOFTWARE IS SUPPLIED BY MICROCHIP "AS IS". NO WARRANTIES, WHETHER EXPRESS,
# IMPLIED OR STATUTORY, APPLY TO THIS SOFTWARE, INCLUDING ANY IMPLIED WARRANTIES
# OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE. IN
# NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL
# OR CONSEQUENTIAL LOSS, DAMAGE, COST OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO
# THIS SOFTWARE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY
# OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP'S
# TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THIS SOFTWARE WILL NOT EXCEED
# THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THIS SOFTWARE.
import json
from base64 import b64decode
from argparse import ArgumentParser
import jose.jws
from jose.utils import base64url_decode, base64url_encode
from cryptography import x509
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import ec, ed25519, ed448, x25519, x448, rsa

parser = ArgumentParser(
    description='Verify and decode secure element manifest'
)
parser.add_argument(
    '--manifest',

```

```

    help='Manifest file to process',
    nargs=1,
    type=str,
    required=True,
    metavar='file'
)
parser.add_argument(
    '--cert',
    help='Verification certificate file in PEM format',
    nargs=1,
    type=str,
    required=True,
    metavar='file'
)
args = parser.parse_args()

# List out allowed verification algorithms for the JWS. Only allows
# public-key based ones.
verification_algorithms = [
    'RS256', 'RS384', 'RS512', 'ES256', 'ES384', 'ES512'
]

# Load manifest as JSON
with open(args.manifest[0], 'rb') as f:
    manifest = json.load(f)

# Load verification certificate in PEM format
with open(args.cert[0], 'rb') as f:
    verification_cert = x509.load_pem_x509_certificate(
        data=f.read(),
        backend=default_backend()
    )

# Convert verification certificate public key to PEM format
verification_public_key_pem = verification_cert.public_key().public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
).decode(encoding='ascii')

# Get the base64url encoded subject key identifier for the verification cert
ski_ext = verification_cert.extensions.get_extension_for_class(
    extclass=x509.SubjectKeyIdentifier
)
verification_cert_kid_b64 = base64url_encode(
    ski_ext.value.digest
).decode(encoding='ascii')

# Get the base64url encoded sha-256 thumbprint for the verification cert
verification_cert_x5t_s256_b64 = base64url_encode(
    verification_cert.fingerprint(hashes.SHA256())
).decode(encoding='ascii')

# Process all the entries in the manifest
for i, signed_se in enumerate(manifest):
    print('')
    print('Processing entry {} of {}'.format(i+1, len(manifest)))
    print('uniqueId: {}'.format(
        signed_se['header']['uniqueId']
    ))

    # Decode the protected header
    protected = json.loads(
        base64url_decode(
            signed_se['protected'].encode('ascii')
        )
    )
    if protected['kid'] != verification_cert_kid_b64:
        raise ValueError('kid does not match certificate value')
    if protected['x5t#S256'] != verification_cert_x5t_s256_b64:
        raise ValueError('x5t#S256 does not match certificate value')

    # Convert JWS to compact form as required by python-jose
    jws_compact = '.'.join([
        signed_se['protected'],
        signed_se['payload'],
        signed_se['signature']
    ])

```



```

# Verify and decode the payload. If verification fails an exception will
# be raised.
se = json.loads(
    jose.jws.verify(
        token=jws_compact,
        key=verification_public_key_pem,
        algorithms=verification_algorithms
    )
)
if se['uniqueId'] != signed_se['header']['uniqueId']:
    raise ValueError(
        (
            'uniqueId in header "{}" does not match version in' +
            ' payload "{}"'
        ).format(
            signed_se['header']['uniqueId'],
            se['uniqueId']
        )
    )
print('Verified')

print('SecureElement = ')
print(json.dumps(se, indent=2))

# Decode public keys and certificates
try:
    public_keys = se['publicKeySet']['keys']
except KeyError:
    public_keys = []

for jwk in public_keys:
    print('Public key in slot {}'.format(int(jwk['kid'])))

    if jwk['kty'] == 'EC':
        # Decode x and y integers
        x = int.from_bytes(bytes=base64url_decode(jwk['x'].encode('utf8')),
            byteorder='big')
        y = int.from_bytes(bytes=base64url_decode(jwk['y'].encode('utf8')),
            byteorder='big')

        # Get public key
        if jwk['crv'] == 'P-224':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.SECP224R1(), x=x,
                y=y).public_key(
                    backend=default_backend()
                )
        elif jwk['crv'] == 'P-256':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.SECP256R1(), x=x,
                y=y).public_key(
                    backend=default_backend()
                )
        elif jwk['crv'] == 'P-384':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.SECP384R1(), x=x,
                y=y).public_key(
                    backend=default_backend()
                )
        elif jwk['crv'] == 'P-521':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.SECP521R1(), x=x,
                y=y).public_key(
                    backend=default_backend()
                )
        elif jwk['crv'] == 'secp256k1':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.SECP256K1(), x=x,
                y=y).public_key(
                    backend=default_backend()
                )
        elif jwk['crv'] == 'brainpoolP256r1':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.BrainpoolP256R1(), x=x,
                y=y).public_key(
                    backend=default_backend()
                )
        elif jwk['crv'] == 'brainpoolP384r1':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.BrainpoolP384R1(), x=x,
                y=y).public_key(
                    backend=default_backend()
                )
        elif jwk['crv'] == 'brainpoolP512r1':
            public_key = ec.EllipticCurvePublicNumbers(curve=ec.BrainpoolP512R1(), x=x,

```

```

y=y).public_key(
    backend=default_backend()
)
else:
    raise ValueError(
        'Unsupported {}'.format(json.dumps({'crv': jwk['crv']}))
    )

print(public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
).decode(encoding='ascii'))

elif jwk['kty'] == 'OKP':
    if jwk['crv'] == 'Ed25519':
        public_key =
ed25519.Ed25519PublicKey.from_public_bytes(data=base64url_decode(jwk['x']))
    elif jwk['crv'] == 'Ed448':
        public_key =
ed448.Ed448PublicKey.from_public_bytes(data=base64url_decode(jwk['x']))
    elif jwk['crv'] == 'X25519':
        public_key =
x25519.X25519PublicKey.from_public_bytes(data=base64url_decode(jwk['x']))
    elif jwk['crv'] == 'X448':
        public_key =
x448.X448PublicKey.from_public_bytes(data=base64url_decode(jwk['x']))
    else:
        raise ValueError(
            'Unsupported {}'.format(json.dumps({'crv': jwk['crv']}))
        )

print(public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
).decode(encoding='ascii'))

elif jwk['kty'] == 'RSA':
    n = int.from_bytes(bytes=base64url_decode(jwk['n']).encode('utf8')),
byteorder='big')
    e = int.from_bytes(bytes=base64url_decode(jwk['e']).encode('utf8')),
byteorder='big')

    public_key = rsa.RSAPublicNumbers(n=n, e=e).public_key(backend=default_backend())

print(public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
).decode(encoding='ascii'))

else:
    raise ValueError(
        'Unsupported {}'.format(json.dumps({'kty': jwk['kty']}))
    )

# Decode any available certificates
for cert_b64 in jwk.get('x5c', []):
    cert = x509.load_der_x509_certificate(
        data=b64decode(cert_b64),
        backend=default_backend()
    )

print(cert.public_bytes(
    encoding=serialization.Encoding.PEM
).decode(encoding='ascii'))

```

4. Revision History

Doc Rev.	Date	Description
B	07/2025	Added support for the following asymmetric public keys: • Elliptic Curves – NIST P-224, P-384, P-521 – Secp256k1 (Bitcoin/Blockchain) – Brainpool 256-bit, 384-bit, 512-bit • Octet Key Pairs – Ed25519, Ed448 – X25519, X448 • RSA – 2048-bit, 3072-bit, 4096-bit
A	02/2022	Initial release of this document

Microchip Information

Trademarks

The “Microchip” name and logo, the “M” logo, and other names, logos, and brands are registered and unregistered trademarks of Microchip Technology Incorporated or its affiliates and/or subsidiaries in the United States and/or other countries (“Microchip Trademarks”). Information regarding Microchip Trademarks can be found at <https://www.microchip.com/en-us/about/legal-information/microchip-trademarks>.

ISBN: 979-8-3371-1424-8

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP “AS IS”. MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP’S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer’s risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip products are strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is “unbreakable”. Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.